

CSE445

CSE445 Final Project Presentation

Section : 7

- Shawlin Salit 2212906042
- Ridita Afrin Riya 2211622042
- Mubtasim Fuad 2131135642
- Faiyazul Islam Rimon 2231898042
- Md. Nazibul Islam Nabil 2222456042

Project Introduction & Data Loading

- **Objective:** Classifying 5 distinct animal species (Cat, Cow, Dog, Lamb, Zebra).
- **Data Integrity:** We utilized a perfectly balanced dataset of 500 images (100 per class).
- **Loading Mechanism:** Images were loaded in micro-batches of 32 to prevent memory overflow. I strictly enforced shuffle=False during loading to guarantee that our extracted mathematical features perfectly aligned with their correct animal labels.

Deep Feature Extraction (VGG16)

- **The Problem:** Classical ML models cannot natively understand shapes or fur from raw pixels.
- **Transfer Learning:** I utilized the pre-trained VGG16 Neural Network, removing its classification head to act purely as a spatial scanner.
- **Global Average Pooling:** This mathematical operation compressed every raw image into a highly dense, 512-Dimensional feature vector, preparing the data for classification.

Logistic Regression & Evaluation

- **My Classifier:** I implemented a Multinomial Logistic Regression model to classify the 512-D vectors.
- **Optimization:** To prevent the gradient descent algorithm from crashing in such a high-dimensional space, I hyper-tuned the solver by setting `max_iter=2000`.
- **Evaluation:** My Confusion Matrix proved the efficacy of the VGG16 features, showing near-perfect classification with almost zero catastrophic errors.

Random Forest Ensemble

- **The Problem with Trees:** A single Decision Tree is highly prone to memorizing the training data, leading to severe overfitting.
- **My Classifier:** To solve this, I constructed a Random Forest Ensemble consisting of 100 completely separate, randomized decision trees.
- **The Voting Mechanism:** During testing, all 100 trees look at the 512-D image features and cast a vote. The majority vote wins, which mathematically flattens out variance and errors.

Hyperparameter Tuning (GridSearchCV)

- **Optimization Strategy:** I did not manually guess the best settings for my forest. I implemented GridSearchCV, a brute-force search algorithm.
- **The Grid:** I provided the algorithm with a grid of options, including various tree counts (50, 100, 200) and different branch depths.
- **Cross-Validation:** Utilizing 5-Fold Cross Validation, the algorithm systematically tested every combination and automatically locked in the most mathematically optimal hyperparameters for my model.

Proving No Overfitting (Learning Curves)

- **The Critique:** 98% accuracy on a small dataset often implies the model is just memorizing data.
- **My Defense:** I mathematically disproved this by plotting a Learning Curve. After globally shuffling the data, I trained the model incrementally from 10% up to 100% of the data.
- **The Proof:** Because my Orange Validation Line perfectly converged with my Blue Training Line, it proves my Random Forest generalized perfectly to new data and did not overfit.

K-Nearest Neighbors (KNN)

- **My Classifier:** Instead of calculating complex statistical boundaries, I implemented a highly robust, non-parametric algorithm called K-Nearest Neighbors (KNN).
- **The Logic:** KNN does not attempt to draw lines between classes. Instead, it relies purely on the spatial grouping of the 512-Dimensional features extracted by VGG16.

Distance Metrics in High Dimensions

- **How it Works:** When trying to classify a brand new, unseen image, my algorithm plots it into a massive mathematical space.
- **Euclidean Distance:** It calculates the exact Euclidean distance to all the other training images. It then classifies the new animal based on a majority vote from its closest mathematical “neighbors”. Because VGG16 groups similar animals tightly together, KNN is incredibly accurate.

Classification Report Metrics

- **Advanced Evaluation:** I evaluated my model using a rigorous Classification Report rather than just a basic accuracy percentage.
- **Precision and Recall:** I calculated Precision (exactness of the predictions) and Recall (completeness of finding the animals).
- **The Result:** My model scored between 0.98 and 1.00 across all 5 animals, proving my KNN algorithm had absolutely zero class-imbalance bias.

Extreme Gradient Boosting (XGBoost)

- **My Classifier:** I utilized XGBoost, an advanced ensemble technique that builds decision trees sequentially.
- **The Logic:** Unlike standard Random Forests that build trees independently, each subsequent tree in my XGBoost model specifically targets and mathematically minimizes the residual pseudo-errors of the preceding tree.

Mathematical Optimization & Configuration

- **Objective Function:** The algorithm approximates the objective function via a second-order Taylor expansion, utilizing the first and second-order gradient statistics of the loss function.
- **Optimization Strategy:** I specifically configured the model to use the mlogloss evaluation metric, which served as a highly aggressive, optimized boosting approach for multi-class categorization of the 512-D VGG16 features.

Evaluation & Overfitting Analysis

- **Confusion Matrix:** The spatial visualization of errors showed near-perfect diagonals with absolutely zero catastrophic misclassifications (e.g., confusing a Zebra for a Cat).
- **Learning Curve Proof:** By globally randomizing the dataset and plotting incrementally, my validation accuracy curve perfectly converged with the training curve, mathematically proving that XGBoost successfully generalized and completely avoided overfitting.

Hybrid Soft-Voting Classifier

- **My Classifier:** I was responsible for creating the ultimate “Super Model”. I constructed a unified Hybrid Voting Classifier.
- **The Architecture:** My model absorbs multiple base classifying algorithms and unifies their predictive power into a single ensemble mechanism.

The Soft Voting Mechanism

- **Hard vs. Soft Voting:** Hard voting simply tallies up the final categorical guesses of the base models. I explicitly configured my ensemble to use Soft Voting instead.
- **Extracting Probabilities:** My model runs the `.predict_proba()` function on all base algorithms to extract their exact mathematical confidences (e.g., 99% sure it's a Zebra, but only 80% sure).
- **The Decision:** It averages all of these probabilities together and selects the absolute most confident answer, drastically reducing extreme errors.

Project Conclusion

- **Computational Efficiency:** Our project proves that you do not need supercomputers or massive datasets to perform advanced computer vision.
- **State-of-the-Art Accuracy:** By combining Deep Transfer Learning (VGG16) with highly-tuned classical algorithms and Soft Ensemble Voting, we achieved near-perfect accuracy while completely avoiding the pitfalls of overfitting.